

Nextjs에서 사용할 수 있는 환경 변수 암호화

Jasypt를 nextjs에서 사용해보자

Sang youn Park

목차

1. 발표 배경
2. Jasypt란
3. 기대효과
4. 구현해보기
 - 4.1.암호화 키 생성
 - 4.2.암복호화 파일 생성
 - 4.3.변수 암호화
 - 4.4.자동 복호화
 - 4.4.1.‘.env’만 사용할 경우
 - 4.4.2.배포환경별로 다를경우(‘.env.local’ 등을 같이 사용할 경우)
 - 4.5.적용 후 발생할 수 있는 이슈
5. Qna

1. 발표배경

- 입사하고 가장 당황스럽게 한 내용

```
$ .env.local
1 DATABASE_URL=postgresql://[redacted]:[redacted]@localhost:5432/[redacted]
2 NEXT_PUBLIC_ENCRYPT_KEY=[redacted]
3 NEXT_PUBLIC_S3_URL=https://[redacted].s3.amazonaws.com/[redacted]
4 BOT_AUTHORIZATION_TOKEN=[redacted]
5 AEGIS_MAIL_API=https://[redacted].aegis.co.kr/api/mail
6
7 NEXT_PUBLIC_SERVICE_DOMAIN=[redacted].vercel.app
8 NEXT_PUBLIC_MAP=[redacted]
9 CENTER_CODE = 0
10
11 RECOIL_DUPLICATE_ACTION_CHECKING_ENABLE=[redacted]
```

- 평문으로 되어있는 속성값을 암호화 해서 조금이라도 유출을 막을 수 있을까 싶어서 발표를 계획함

2. Jasypt(Java Simplified Encryption) 란

- Java 애플리케이션을 위한 암호화 라이브러리
- 단방향, 양방향 암호화를 모두 지원
- 간단한 설정으로 빠른 적용 가능
- 속성값 암호화에 특화됨
- 사용법이 직관적이고 쉬움
- ENC() 함수로 암호화된 값 사용

3. 기대효과

- 보안성 강화
 - 민감한 환경 변수 보호
 - 키값 노출 방지
- 안전한 정보 공유
 - 버전관리 시스템에서 안전하게 관리
 - 암호화된 상태로 배포 가능
- 실수방지
 - 소스 코드 공개 시 정보 유출 방지

4. 구현해보기

4.1. 암호화 키(key) 생성

```
import crypto from 'crypto';

// 32바이트 암호화 키 생성
const generateEncryptionKey = () => crypto.randomBytes(32).toString('hex');
💡

// 16바이트 IV 생성
const generateIV = () => crypto.randomBytes(16).toString('hex');

// 생성된 ENCRYPTION_KEY와 IV 출력
const encryptionKey = generateEncryptionKey();
const iv = generateIV();

console.log(`ENCRYPTION_KEY=${encryptionKey}`);
console.log(`IV=${iv}`);
```

4. 구현해보기

4.1. 암호화 키(key) 생성

- 만들어진 ENCRYPTION_KEY와 IV를 OS 환경변수에 등록한다.
- OS환경변수 이름규칙
 - 대문자 사용
 - 숫자 포함 가능
 - 밑줄 사용
 - 특수 문자 금지

4. 구현해보기

4.2. 암호화 파일 생성

```
import crypto from 'crypto';

// 환경변수에서 암호화 키와 IV를 가져옴.
const ENCRYPTION_KEY = process.env.UIW_ENCRYPTION_KEY; // 32바이트 (AES-256)
const IV = process.env.UIW_IV; // 16바이트

// 암호화 함수
const encrypt = (text) => {
  const cipher = crypto.createCipheriv('aes-256-cbc', Buffer.from(ENCRYPTION_KEY, 'hex'), Buffer.from(IV, 'hex'));
  let encrypted = cipher.update(text, 'utf8', 'hex');
  encrypted += cipher.final('hex');
  return encrypted;
}

// 복호화 함수
export const decrypt = (encryptedText) => {
  const decipher = crypto.createDecipheriv('aes-256-cbc', Buffer.from(ENCRYPTION_KEY, 'hex'), Buffer.from(IV, 'hex'));
  let decrypted = decipher.update(encryptedText, 'hex', 'utf8');
  decrypted += decipher.final('utf8');
  return decrypted;
}
```


4. 구현해보기

4.3 환경변수 암호화

- 암호화 파일 하단에 아래와 같이 등록한 후 파일을 실행한다.

```
const targetStr = '12345678';  
console.log(encrypt(targetStr));
```

```
Syp ~/Developer/workspace/upload_image_to_webp main node commons/utils/encrypto.js  
f8dcedf212824f94e90653840d49578c
```

- 암호화된 문자열을 ENC()에 감싸서 환경변수 파일(.env)에 넣어준다.

```
AWS_ACCESS_KEY_ID=ENC(1179a3b8e96ca277d2721e69a30e862ce66ab4a6e6c9b2d4db9e64b50ead796c)  
AWS_SECRET_ACCESS_KEY=ENC(f05fe214f5eb54aa911184a43030d4d21e844b3259d7e14bbbedf3eaf99231987083fae828230b249c223a81d11ee942c)  
AWS_REGION=ap-northeast-2
```

4. 구현해보기

4.4. 자동복호화('.env'만 사용할 경우)

- next.config.ts에 아래내용을 추가한다.

```
import * as dotenv from 'dotenv';
import * as fs from 'fs';

import {decrypt} from '@utils/encrypt.js'

// 변수값 복호화
const loadDecryptedEnv = () => {
  const envConfig = dotenv.parse(fs.readFileSync('.env'));

  for (const key in envConfig) {
    if (envConfig[key].startsWith('ENC(') && envConfig[key].endsWith('')) { // 암호화 값만 처리
      const encryptedValue = envConfig[key].slice(4, -1); // ENC() 제거
      envConfig[key] = decrypt(encryptedValue);
    }
  }

  return envConfig;
};

const decryptedEnv = loadDecryptedEnv(); //복호화 된 객체

const nextConfig: NextConfig = {
  env: {
    ...decryptedEnv //복호화 값을 env에 다시 저장
  }
};
```

4. 구현해보기

4.4. 자동복호화(배포환경별로 다를경우) - 1

- 'dotenv.config()'를 사용한다.

```
import * as dotenv from 'dotenv';

import {decrypt} from '@utils/encrypt.js'

// .env 파일 로드
dotenv.config();

// 변수값 복호화
const loadDecryptedEnv = () => {
  const decryptedEnv: { [key: string]: string | undefined } = {};
  // 암호화된 환경변수 필터링 및 복호화
  for (const key in process.env) {
    const value = process.env[key];
    if (value && value.startsWith('ENC(') && value.endsWith('')) {
      const encryptedValue = value.slice(4, -1); // ENC() 제거
      decryptedEnv[key] = decrypt(encryptedValue);
    }
  }
  return decryptedEnv;
};

const decryptedEnv = loadDecryptedEnv(); //복호화 된 객체
const nextConfig: NextConfig = {
  env: {
    ...decryptedEnv //복호화 값을 env에 다시 저장
  }
};
```


4. 구현해보기

4.4. 자동복호화(배포환경별로 다를경우 - 2

- 주의사항
 - 'dotenv.config()'를 사용한다.
 - 이때 시스템 상의 모든 환경변수를 가져오게 되는데, react에서 다시 세팅 되는걸 막는 변수들이 있어서 주의할 필요가 있다.
 - 그래서 별도로 복호화 할 key들을 설정해서 하는 방법도 있다.

4. 구현해보기

4.4. 자동복호화(배포환경별로 다를경우) - 3

```
import * as dotenv from 'dotenv';

import {decrypt} from '@utils/encrypt.js'

// .env 파일 로드
dotenv.config();

// 허용할 환경변수 키 목록
const allowedKeys = ['AWS_ACCESS_KEY_ID', 'AWS_SECRET_ACCESS_KEY'];

// 변수값 복호화
const loadDecryptedEnv = () => {
  const decryptedEnv: { [key: string]: string | undefined } = {};
  // 암호화된 환경변수 필터링 및 복호화
  for (const key of allowedKeys) {
    const value = process.env[key];
    if (value && value.startsWith('ENC(') && value.endsWith(')')) {
      const encryptedValue = value.slice(4, -1); // ENC() 제거
      decryptedEnv[key] = decrypt(encryptedValue);
    }
  }
  return decryptedEnv;
};

const decryptedEnv = loadDecryptedEnv(); //복호화 된 객체
const nextConfig: NextConfig = {
  env: {
    ...decryptedEnv //복호화 값을 env에 다시 저장
  }
};
```

4. 구현해보기

4.5 적용 후 발생할 수 있는 이슈

- 키를 분실하지 않도록 주의를 해야한다.
- 라이브러리 자체적으로 환경변수에 접근할 경우엔 복호화 안된값을 참조할 수 있다.
- 이 경우엔 직접 값을 적용해 줘야한다.

```
// AWS S3 설정
const AWS_ACCESS_KEY_ID = process.env.AWS_ACCESS_KEY_ID;
const AWS_SECRET_ACCESS_KEY = process.env.AWS_SECRET_ACCESS_KEY;
const BUCKET_NAME = process.env.S3_BUCKET_NAME; // 환경 변수로 설정된 버킷 이름
const s3 = new S3Client({
  region: process.env.AWS_REGION,
  credentials: {
    accessKeyId: AWS_ACCESS_KEY_ID,
    secretAccessKey: AWS_SECRET_ACCESS_KEY,
  }
});
```

QnA

감사합니다😊